

Ядро знаний Research Knowledge Core

Serving-слой базы знаний CryptoNavigator: один TS-контракт и два рантайма — локально на Windows и на Cloudflare Workers. Гибридный поиск FTS5 + bge-m3, D1 и Vectorize, MCP-инструменты для агентов, пишущих торговых ботов, фазировка с критериями приёмки и верифицированные факты платформы.

CryptoNavigator

приложение для обучения крипто-алготрейдингу · платформа AI-агентов

Содержание

1 • Резюме и статус документа	1
2 • Цели, принципы и границы v2	2
2.1. Что наследуется из v1 без изменений	2
2.2. Что добавляет v2	2
2.3. Границы	2
3 • Обзор архитектуры	3
4 • Компоненты системы	5
4.1. Монорепозитарий	5
4.2. Knowledge Pack и размещение данных	6
4.3. Serving Core (@kc/core)	6
4.4. Адаптеры и потребители	7
5 • Схема данных knowledge pack	7
5.1. Идентификаторы и денормализация	7
5.2. Граф в SQL	7
5.3. FTS5 с преднормализацией	8
6 • Гибридный ретривал	8
6.1. Лексический слой: FTS5 (BM25)	10
6.2. Векторный слой: bge-m3, 1024 измерений	11
6.3. Fusion, фильтры и namespaces	11
6.4. Реранк: четыре режима, честная маркировка	12
6.5. Сборка контекста: Proof-of-Source + Zero-Hallucination	13
7 • Контракты API: REST и MCP	13
7.1. REST (OpenAPI 3.1, /api/k/*)	13
7.2. MCP-контракт (stdio + streamable HTTP)	14
7.3. Интеграция AI-Ментора	16
8 • Стек: решения и верификация	17
8.1. Верифицированные факты (дата проверки: 29.08.2026)	17
8.2. Решения по рекомендациям второго агента	17
9 • Фазировка	18
Phase 0 — MVP и «битва движков» (2–3 недели)	18
Phase 1 — Облако (2 недели)	18
Phase 2 — Исследовательский контур для агентов (3–4 недели)	19

Phase 3 — Раздел 2 v1: статьи и индустрия (по готовности) 19

10 · Риски и открытые вопросы 19

11 · Часть II. Этапы для автономного агента 20

 Формат отчёта после каждого этапа 21

12 · Приложения 22

 Приложение А. Схема БД 22

 Приложение В. JSON-схемы MCP-инструментов (фрагмент) 22

 Приложение С. Примеры запросов 22

 Приложение D. Миграция PSY_RAG_DATABASE в atoms (ЭТАП 69) 23

 Приложение Е. Источники проверки фактов (29.08.2026) 23

1 · Резюме и статус документа

Спецификация v1 («Задание для автономного агента», ЭТАПЫ 1–65) определяет офлайн-пайплайн обработки книжного корпуса: извлечение, чанкинг, атомы, claims, граф отношений, временная модель, supersession. Этот пайплайн остаётся каноническим и не пересматривается. Чего в v1 нет — так это **serving-слоя**: единого способа запрашивать эту базу знаний из AI-Ментора внутри приложения КриптоНавигатор, из будущих агентов, проектирующих и пишущих торговых ботов, и из любых внешних инструментов. v1 зафиксировала полностью локальный Windows-стек (Neo4j, Qdrant, Python API), который принципиально не переносится на Cloudflare Workers — при том, что приложение уже работает с воркером на Workers AI (панель ментора, MENTOR.ask, живые данные через прокси).

v2 закрывает этот разрыв одним архитектурным принципом:

Пайплайн компилирует артефакт (knowledge pack), артефакт запускается на двух рантаймах.

принцип v2

```
Книги/статьи → BUILD (локально, Python+GPU)
    → knowledge_pack.sqlite — source of truth, канонические таблицы
    → records.ndjson       — Cloudflare Vectorize
    → dl-миграции         — Cloudflare D1
→ SERVING (один TS-контракт, два адаптера)
    → Локально: Node + better-sqlite3 + sqlite-vec + llama.cpp
    → Облако: Worker + D1 + Vectorize + Workers AI
→ ПОТРЕБИТЕЛИ
    → AI-Ментор (приложение)
    → Агенты-разработчики ботов (MCP: stdio + streamable HTTP)
    → REST-клиенты
```

Из принципа следуют три ключевых решения. **Первое** — ингестинг никогда не выполняется в облаке: PyMuPDF, GROBID и эмбединги на GPU остаются локальными, воркер только читает скомпилированный пак. **Второе** — API-контракт один (search / get / trace / compare / find_conflicts / best_practices / validate_claim / check_design — надмножество ЭТАПА 28 и ЭТАПА 63 v1), реализаций две, и потребитель не знает, где исполняется запрос. **Третье** — Neo4j и Qdrant перестают быть зависимостями рантайма: граф живёт в SQL-таблице relations, векторы — в sqlite-vec/Vectorize; Neo4j остаётся опциональным зеркалом для визуальной отладки.

Стек serving-слоя согласован с рекомендациями второго агента проекта (FTS5 вместо рукописного BM25, sqlite-vec локально, канонические записи в таблицах, один TS-слой с двумя тонкими адаптерами, единый search() для ментора и агентов) — с четырьмя поправками, подтверждёнными проверкой фактов от 29.08.2026: ранкер из Workers AI **не мультиязычный** (раздел 6.4), namespaces Vectorize следует использовать **для изоляции сред, а не доменов** (раздел 6.3), FTS5-токенизатор **не стеммит кириллицу** — стемминг переносится в TS-слой (раздел 6.1), а лимит размерности Vectorize составляет **1536**, что вмещает 1024-мерный bge-m3 без усечения (раздел 6.2).

2 · Цели, принципы и границы v2

2.1. Что наследуется из v1 без изменений

Все критические запреты v1 остаются в силе и распространяются на serving-слой: запрет №1 (никогда «книга → LLM summary → embeddings»), запрет №2 (новизна источника — не критерий истины), запрет №3 (SOURCE / DERIVED / APPLICATION / EXPERIMENTAL никогда не смешиваются), запрет №4 (старые источники не удаляются), запрет №5 (выдуманные цитаты запрещены), запрет №12 (temporal_status книжного claim не меняется на основании одной статьи). Provenance-цепочка «RECOMMENDATION → DERIVED CLAIM → SUPPORTING ATOMS → SOURCE CHUNKS → ORIGINAL DOCUMENT» обязательна для любого ответа serving-слоя: если путь невозстановим, утверждение не считается знанием CryptoNavigator.

2.2. Что добавляет v2

Область	v1	v2
Рантайм поиска	Python API, локально	Один TS-контракт: Node локально, Worker в облаке
Лексический поиск	н/д	FTS5 (BM25) в SQLite и D1, общий SQL
Векторный поиск	Qdrant	sqlite-vec (локально) + Vectorize (облако), bge-m3
Реранк	«multilingual reranker»	Конфигурируемый: local / cloud / llm / none
Граф	Neo4j обязательный	SQL-таблица relations + CTE; Neo4j — опция
Доступ агентов	н/д	MCP (stdio + streamable HTTP) + REST
Ментор	н/д	Серверная сборка контекста; клиентский движок — фолбэк
Eval	700 тестов после прототипа	Golden set с Phase 0 + contract-тесты двух адаптеров

Таблица 1 — изменения v2 относительно v1

2.3. Границы

v2 сознательно не включает: ингестинг в облаке (невозможен и не нужен), мониторинг arXiv и quality gate из Раздела 2 v1 (это Phase 3), автоматическое разрешение конфликтов (ЭТАП 60.3: конфликты сохраняются для ручного разбора), генерацию торговых сигналов (запреты приложения сохраняются: фильтр MENTOR.filter остаётся на месте).

3 · Обзор архитектуры

Система состоит из четырёх слоёв, каждый со своей зоной ответственности и своей дисциплиной выпусков: build, pack, serving и потребители. Полная схема — на рис. 1.



Рис. 1 — Общая архитектура: build компилирует knowledge pack; serving исполняется на двух рантаймах за одним контрактом

Слой BUILD (локальный, Python). Реализует ЭТАПЫ 1–20 v1 для книг и ЭТАПЫ 51–60 для статей и отчётов: сканирование корпуса, манифест, дедупликация, извлечение текста (PyMuPDF / ebooklib),

контроль качества, канонический корпус, семантический чанкинг, contextual retrieval, атомы, валидация атомов (Extractor → Validator → Resolver → Conflict модели по ЭТАПУ 45), claims, концепции, отношения, эмбединги (bge-m3 через llama.cpp на локальном GPU — мощности RTX 5070 Ti достаточно с запасом). Выход — не «живая система», а артефакты. Build идемпотентен: повторный запуск по тому же корпусу даёт тот же пак; детерминизм важнее скорости.

Слой PACK (артефакт). knowledge_pack.sqlite — единственный источник истины: канонические таблицы (documents, chapters, sections, chunks, atoms, claims, concepts, relations, lessons, golden_tests, pack_meta) плюс serving-структуры (FTS5-таблицы с преднормализованным текстом; vec0-таблицы локально). Из пака экспортируются: records_ndjson — векторы и метаданные для Vectorize (по индексу на атомы и чанки), d1-миграции для облачной БД (та же схема минус vec0) и graph_export.jsonl — опциональное зеркало для Neo4j Browser. Версия пака записывается в pack_meta и прошивает все ответы serving-слоя.

Слой SERVING (TypeScript, два адаптера). Пакет @kc/core содержит всё, что не зависит от рантайма: нормализацию и стемминг запроса, fusion (RRF), фильтры, пороги релевантности, бонусы урока и слабых зон ученика (перенос логики CryptoMentorRAG v2), сборку Zero-Hallucination контекста, форматы ответов. Поверх — два тонких адаптера: adapter-local (better-sqlite3 + sqlite-vec + llama.cpp) и adapter-cloud (биндинги D1, Vectorize, Workers AI). Адаптер реализует интерфейс SearchBackend: ftsSearch(), vectorSearch(), getAtoms(), getChunks(), rerank() — и ничего больше.

Потребители. Во-первых, AI-Ментор: воркер приложения при action=askBooks вызывает serving core и сам собирает промпт — клиент больше не тащит тысячи атомов в HTML (текущий клиентский движок остаётся офлайн-фолбэком). Во-вторых, агенты-разработчики: MCP-сервер с инструментами knowledge_search, check_design и другими — локально через stdio (Claude Code, Cursor), в облаке через streamable HTTP. В-третьих, REST — тот же контракт для любых не-MCP клиентов.

Ключевое решение — **почему один TS-слой, а не Python-локально и TS-в-облаке**: логика релевантности (пороги, стемминг, бонусы, веса fusion) обязана вести себя идентично на обоих рантаймах, иначе метрики eval расщепятся и «битва движков» потеряет смысл. Дублирование на двух языках — гарантированный дрейф. Python остаётся только в build, где его сила (PyMuPDF, ML-стек) и где нет требований латентности.

4 · Компоненты системы

4.1. Монорепозиторий

структура репозитория

knowledge-core/	
└─ schema/	# 0001_init.sql + миграции
└─ tools/build/	# Python 3.11+ (uv): ЭТАПЫ v1
└─ └─ extractor/	# PyMuPDF, ebooklib, pdfplumber, GROBID (опция)
└─ └─ chunker/	# семантический чанкинг (ЭТАП 7, 57)
└─ └─ atomizer/	# атомы + JSON-схемы промптов (ЭТАП 9, 45)
└─ └─ validator/	# Validator-модель (ЭТАП 10)
└─ └─ graph/	# relations, concepts, conflicts (ЭТАП 16, 19, 32)
└─ └─ embeddings/	# bge-m3 @ llama.cpp /v1/embeddings
└─ └─ exporters/	# pack.sqlite, ndjson, d1-миграции, graph_export
└─ packages/	
└─ └─ core/	# @kc/core – рантайм-независимое ядро
└─ └─ adapter-local/	# better-sqlite3 + sqlite-vec
└─ └─ adapter-cloud/	# D1 / Vectorize / Workers AI
└─ apps/	
└─ └─ worker/	# Cloudflare Worker: REST + MCP streamable HTTP
└─ └─ mcp-local/	# MCP stdio (Claude Code, Cursor)
└─ └─ api-local/	# Node (Hono): локальный REST
└─ evals/	# golden set + раннер + «битва движков»
└─ docs/	# ARCHITECTURE.md и др. (ЭТАП 49 v1)

4.2. Knowledge Pack и размещение данных

Данные	Локально	Облако
Канонические таблицы	knowledge_pack.sqlite	D1 knowledge_core (та же схема)
Полный текст чанков	chunks.text	D1 chunks.text (лимит 10 GB — см. риски)
FTS5-индексы	в том же файле	в D1 (поддерживается)
Векторы атомов/чанков	vec0-таблицы в файле	Vectorize kc-atoms-prod / kc-chunks-prod, 1024d
Эмбеddинг запроса	llama.cpp /v1/embeddings	Workers AI @cf/baai/bge-m3
Реранк	llama.cpp /v1/rerank (v2-m3)	конфигурируемый — раздел 6.4
Граф	relations + рекурсивные CTE	те же таблицы в D1
Neo4j (опция)	зеркало graph_export.jsonl	н/д

Таблица 2 — размещение данных между рантаймами

Модель выпуска: build → локальная валидация (contract-тесты + golden set) → экспорт → деплой D1-миграций и Vectorize upsert → канареечное переключение namespace → prod. Откат — возврат на предыдущий pack_version: источник истины всегда локальный пак, D1 пересобирается миграциями, а не ручными правками.

4.3. Serving Core (@kc/core)

- **normalizeQuery(q)** — нижний регистр, ё→е, снежинкостемминг RU и EN; тот же нормализатор, что в build;
- **search(params)** — оркестрация: fts + vector → RRF → фильтры → порог → реранк → топ-К;

- **buildMentorContext(results, profile)** — сборка Proof-of-Source контекста по правилам CryptoMentorRAG v2 (только по источникам; автор + книга + глава; честное «в базе нет»; различие позиций автора и протокола курса; подпись «пересказ, не дословная цитата» при quote_verified=0);
- **trace / compare / findConflicts / bestPractices / validateClaim / checkDesign** — исследовательские методы ЭТАПОВ 30–35 v1;
- **composeNoHitsAnswer()** — ветка Zero-Hallucination (порог Д-1 v10.8: слабое совпадение одного токена не считается находкой).

Контракт ответа един для всех методов: results, pack_version, backend (local|cloud), rerank (local|cloud|llm|none), took_ms — потребитель всегда видит, каким рантаймом и с каким качеством ранжирования получен ответ.

4.4. Адаптеры и потребители

adapter-local: better-sqlite3 (файл открывается read-only), sqlite-vec через loadExtension, HTTP-клиент llama.cpp. Обслуживает apps/mcp-local и apps/api-local. **adapter-cloud:** биндинги env.DB, env.VEC_ATOMS / env.VEC_CHUNKS, env.AI; обслуживает apps/worker; вся логика наследуется из @kc/core, цель адаптера — менее 300 строк обёрток. **Ментор:** MENTOR.ask отправляет только вопрос и lessonId, воркер сам вызывает serving core и вставляет контекст в системный промпт; требования Д-6 v10.8 (whitelist действий, каналы __rag/__recent/__triage) сохраняются. **Агенты:** полный доступ к исследовательским методам через MCP — здесь реализуется «архитектура с упором на агентов»: агент-конструктор обязан проходить цикл ЭТАПА 34 v1 (идея → поиск → практики → контрпримеры → конфликты → дизайн → валидация → код → бэктест → риск-ревью), а не «LLM → код».

5 · Схема данных knowledge pack

Полный DDL — в файле schema/0001_init.sql (приложение А); схема проверена применением к чистому SQLite: 25 объектов, ноль ошибок. Ниже — ключевые решения.

5.1. Идентификаторы и денормализация

Идентификаторы — строковые с префиксами типов (doc_, c_, at_, cl_, cn_, rel_, gt_), чтобы ссылки в relations и векторы Vectorize были самодостаточными. Существующие атомы психологии мигрируют с сохранением id (psy:tnd_001) — ссылки из PSY_RAG_DATABASE и старых логов ментора остаются валидными.

Денормализация атомов. Таблица atoms объединяет канонические поля v1 (statement, atom_type, source_refs, validation_status, confidence, knowledge_level) и поля карточки ментора из PSY_RAG v4.2 (topic, core_idea, author_case, step_by_step_protocol, verbatim_anchor_quote, quote_verified, linked_lessons). Решение осознанное: knowledge pack — скомпилированный артефакт для serving, а не транзакционная БД билда; одна таблица — один запрос сборки карточки без джойнов на горячем пути. Билд остаётся нормализованным в своих JSONL (ЭТАП 6 v1).

5.2. Граф в SQL

relations(subject_type, subject_id, predicate, object_type, object_id, rationale) — единая таблица рёбер, покрывающая ЭТАПЫ 16, 23, 58 и 62 v1 (supports / contradicts / supersedes / cites и другие). Иерархия provenance — через внешние ключи (atoms.chunk_id → chunks → chapters → documents), поэтому trace() — два плоских запроса, а обход графа цитирования — рекурсивный СТЕ:

обход графа цитирования

```
WITH RECURSIVE cites(id, depth) AS (
  SELECT object_id, 0 FROM relations
  WHERE subject_id = :doc_id AND predicate IN ('cites', 'uses_method_from')
  UNION ALL
  SELECT r.object_id, c.depth+1 FROM relations r
  JOIN cites c ON r.subject_id = c.id WHERE c.depth < 2
)
SELECT DISTINCT id FROM cites;
```

5.3. FTS5 с преднормализацией

Виртуальные таблицы chunks_fts и atoms_fts хранят уже нормализованный текст (нижний регистр + стемминг), а не оригинал — оригинал неизменен в chunks.text (ЭТАП 8 v1). Запрос нормализуется тем же кодом в TS. Причина: токенизатор unicode61 корректно режет кириллицу по словам, но не стеммит («позиция» ≠ «позиции» ≠ «позиций»), а кастомные токенизаторы в D1 недоступны. Стемминг в TS-слое (snowball ru+en, общий словарь суффиксов для build-Python и core-TS) решает это одинаково на обоих рантаймах.

Vectorize-метаданные вектора: ref_id, domain, author, doc_type, source_class, knowledge_level, temporal_status, evidence_level, lang, lesson_id, year — 10 индексируемых полей (лимит Vectorize), значения не длиннее 64 байт; расширенные списки (domains[], lesson_ids[], keywords[]) кладутся в метаданные без индексации (лимит 10 KiB на вектор) и дофильтровываются в D1. **pack_meta** хранит конфиг ретривала: версии, модель и размерность эмбединга, пороги (relevance_threshold = 1.5 из Д-1), rrf_k, конфиг перанкера, имена индексов и namespaces. **golden_tests** — golden set ЭТАПА 42 v1 переносится вместе с данными, поэтому eval-раннер всегда согласован с версией корпуса.

6 · Гибридный ретривал

Путь данных от исходников к артефактам показан на рис. 2, путь запроса в serving-слое — на рис. 3.

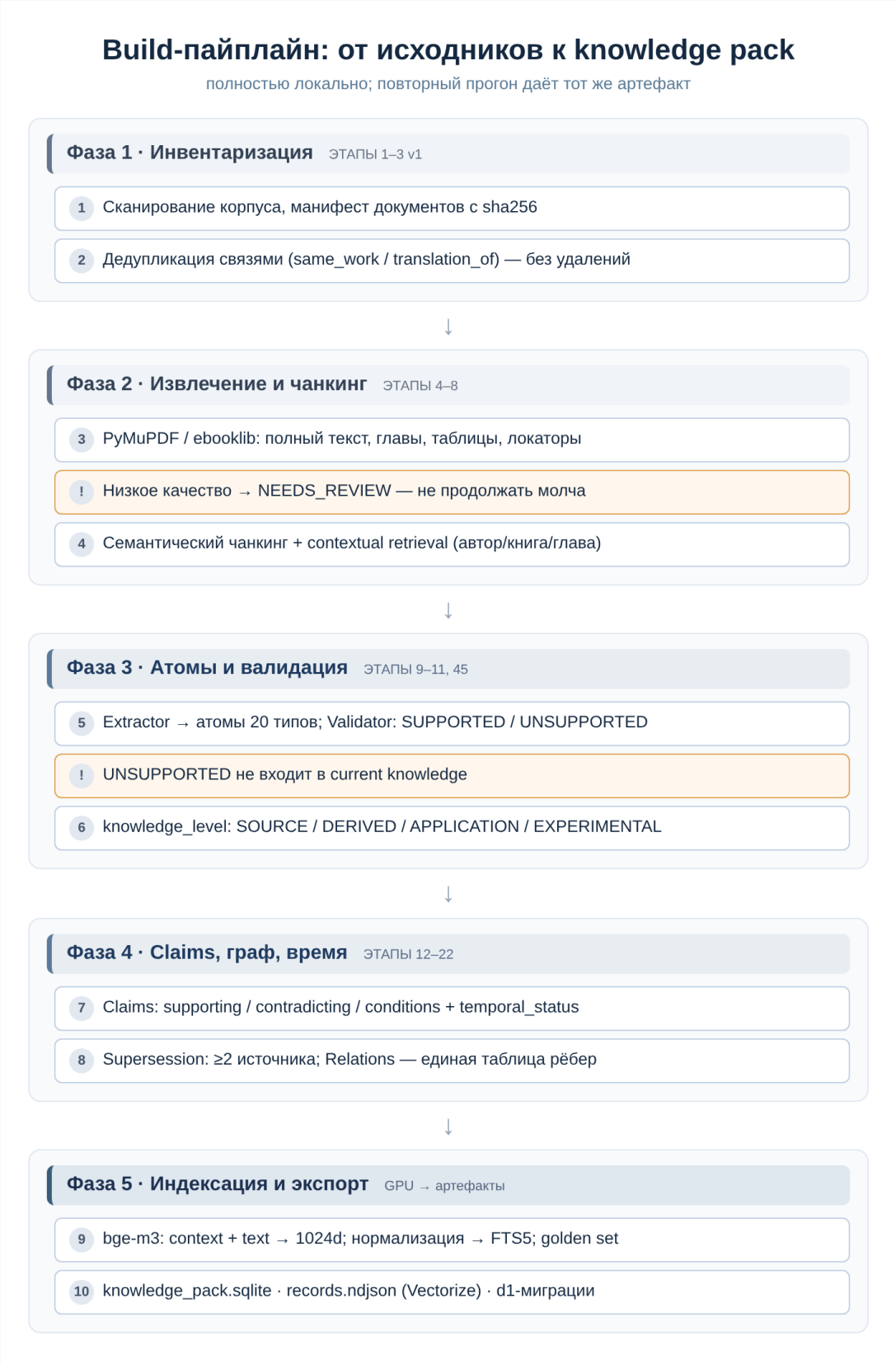


Рис. 2 — Build-пайплайн: от исходников к knowledge pack (локально, Python + GPU)



Рис. 3 — Путь запроса: нормализация → параллельный лексический и векторный поиск → RRF → фильтры → реранк → сборка контекста

6.1. Лексический слой: FTS5 (BM25)

FTS5 доступен и в локальном SQLite, и в D1 (подтверждено 29.08.2026, раздел 8) — один и тот же SQL работает в обоих рантаймах, рукописный JS-BM25 из index_v10.8 в serving-слое не воспроизводится:

FTS5-запрос (одинаков для SQLite и D1)

```
SELECT a.atom_id, bm25(atoms_fts) AS score
FROM atoms_fts f JOIN atoms a ON a.atom_id = f.atom_id
WHERE atoms_fts MATCH :normalized_query
ORDER BY score LIMIT 30;
```

Нормализация запроса и документа — единый `normalize()` из `@kc/core`. Весовые коэффициенты полей (тема и теги тяжелее текста, как в движке v10.2) реализуются дублированием ключевых полей в `norm_text` атома — приём уже используется в CryptoMentorRAG.

6.2. Векторный слой: bge-m3, 1024 измерений

Модель — BAAI/bge-m3: мультязычная (русский ↔ английский ↔ смешанный корпус), длинный контекст. Размерность 1024 укладывается в лимит Vectorize 1536 без усечения и сжатия. Локально эмбединги считает llama.cpp (GGUF bge-m3) — на RTX 5070 Ti это сотни чанков в секунду; в облаке — @cf/baai/bge-m3 (\$0.012 за миллион входных токенов). Эмбединится не «голый» чанк, а `context_block + text` (contextual retrieval, ЭТАП 8 v1) — это резко улучшает поиск по фразе «что говорит Тендлер про гнев».

Два Vectorize-индекса: `kc-atoms-prod` (карточки для ментора) и `kc-chunks-prod` (полный корпус). Разделение оправдано разными паттернами: ментор почти всегда ищет атомы, исследовательский агент — чанки; отдельные индексы не размазывают top-K.

6.3. Fusion, фильтры и namespaces

Слияние — Reciprocal Rank Fusion: $\text{score}(d) = \sum 1/(k + \text{rank}_i(d))$, $k = 60$ (конфиг в `pack_meta`). RRF выбран вместо взвешанной суммы нормализованных скоров: он не требует калибровки шкал BM25 и косинусной близости и стабилен при смене модели эмбединга.

Поле	Тип	Индекс	Пример фильтра
domain	string	да	\$eq: "psychology"
author	string	да	\$in: ["Jared Tendler", "Mark Douglas"]
doc_type	string	да	book / paper_preprint / ...
source_class	string	да	A–E
knowledge_level	string	да	SOURCE / DERIVED / ...
temporal_status	string	да	current / superseded / historical
evidence_level	string	да	A–E
lang	string	да	en / ru
lesson_id	string	да	ps_l20 (primary_lesson)
year	number	да	\$gte: 2015

Таблица 3 — метаданные вектора Vectorize: 10 индексируемых полей (лимит)

Namespaces — для сред, не для доменов. Запросы ментора и агентов обычно кросс-доменные («психология риска + позиционирование»), а namespace в Vectorize — жёсткая партиция: вектор принадлежит ровно одному namespace, кросс-namespace поиска нет. Поэтому namespace используется для изоляции prod/dev, а домены остаются в metadata-фильтрах, где \$in-запросы свободно пересекаются. Лимит в 50 000 namespaces на индекс (повышен в апреле 2026 с 100) проблемы не составляет.

6.4. Реранк: четыре режима, честная маркировка

Проверка 29.08.2026 выявила критичное ограничение: @cf/baai/bge-reranker-base — не мультиязычная модель (обучена на EN-парах; запрос на bge-reranker-v2-m3 открыт с апреля 2025). Для русских запросов ментора она ненадёжна, поэтому реранк — конфиг с четырьмя режимами в pack_meta.reranker_config:

Режим	Реализация	Когда применяется
local	llama.cpp /v1/rerank + bge-reranker-v2-m3 GGUF (мультиязычный)	локальный рантайм — всегда
cloud	@cf/baai/bge-reranker-base	облако, запрос заведомо EN (детект языка)
llm	Workers AI llama-3.3-70b: ранжирование пар → JSON	облако, RU-запросы в рамках p95-бюджета
none	RRF-топ как есть	фолбэк и A/B-базлайн прироста реранка

Таблица 4 — режимы реранка

Выбор режима прошивается в ответ — ментор и агенты видят качество ранжирования, а метрика Recall@5 считается отдельно по каждому режиму на golden set: «реранкер не фиксируется навсегда» (ЭТАП 27 v1).

6.5. Сборка контекста: Proof-of-Source + Zero-Hallucination

После топ-5...8 собирается блок доказательной базы по правилам CryptoMentorRAG v2 с расширениями v2: каждый источник помечается knowledge_level и temporal_status, у конфликтных утверждений указываются contradicting_sources (ЭТАП 32: различать REAL_CONTRADICTION и DIFFERENT_CONDITIONS). Порог релевантности Д-1 сохранён: при нуле находок — честная ветка «в базе знаний по этому вопросу конкретных указаний нет», и каждый такой промах логируется как кандидат в golden set — живой поток запросов учеников это редчайший актив для eval.

7 · Контракты API: REST и MCP

7.1. REST (OpenAPI 3.1, /api/k/*)

Метод и путь	Назначение	Аналог v1
POST /api/k/search	гибридный поиск атомов и чанков	ЭТАП 28–29 search()
POST /api/k/get	карточки по идентификаторам	ЭТАП 28 get()
GET /api/k/trace?ref=	provenance-путь до книги и страницы	ЭТАП 30 trace()
POST /api/k/compare	сравнение авторов по концепции	ЭТАП 31 compare()
POST /api/k/conflicts	конфликты и их классификация	ЭТАП 32 find_conflicts()
POST /api/k/best-practices	current / foundational / historical / deprecated	ЭТАП 33 get_best_practices()
POST /api/k/validate	вердикт по утверждению с источниками	ЭТАП 28 validate_claim()
POST /api/k/check-design	16 проверок торговой системы	ЭТАП 35 check_design()
POST /api/mentor/context	сборка контекста урока для ментора	ЭТАП 36 mentor mode

Таблица 5 — REST-эндпоинты serving-слоя

Пример запроса поиска (совместимость с ЭТАПОМ 29 v1 сохраняется):

пример search

```
POST /api/k/search
{
  "query": "как выбирать размер позиции при высокой неопределенности",
  "mode": "atoms",
  "domains": ["risk", "portfolio"],
  "knowledge_types": ["principle", "constraint", "failure_mode"],
  "asset_classes": ["crypto"],
  "temporal_status": "current",
  "top_k": 10
}

→ 200 {
  "results": [{
    "atom_id": "psy:tnd_042", "atom_type": "principle",
    "knowledge_level": "SOURCE", "temporal_status": "current",
    "evidence_level": "C",
    "score": {"bm25": 8.1, "vector": 0.78, "rrf": 0.031},
    "provenance": {"book": "The Mental Game of Trading",
      "author": "Jared Tendler", "chapter_num": 7,
      "chunk_id": "c_doc_000017_000341",
      "quote_verified": false}
  }],
  "pack_version": "2.0.0", "backend": "cloud",
  "rerank": "none", "took_ms": 214
}
```

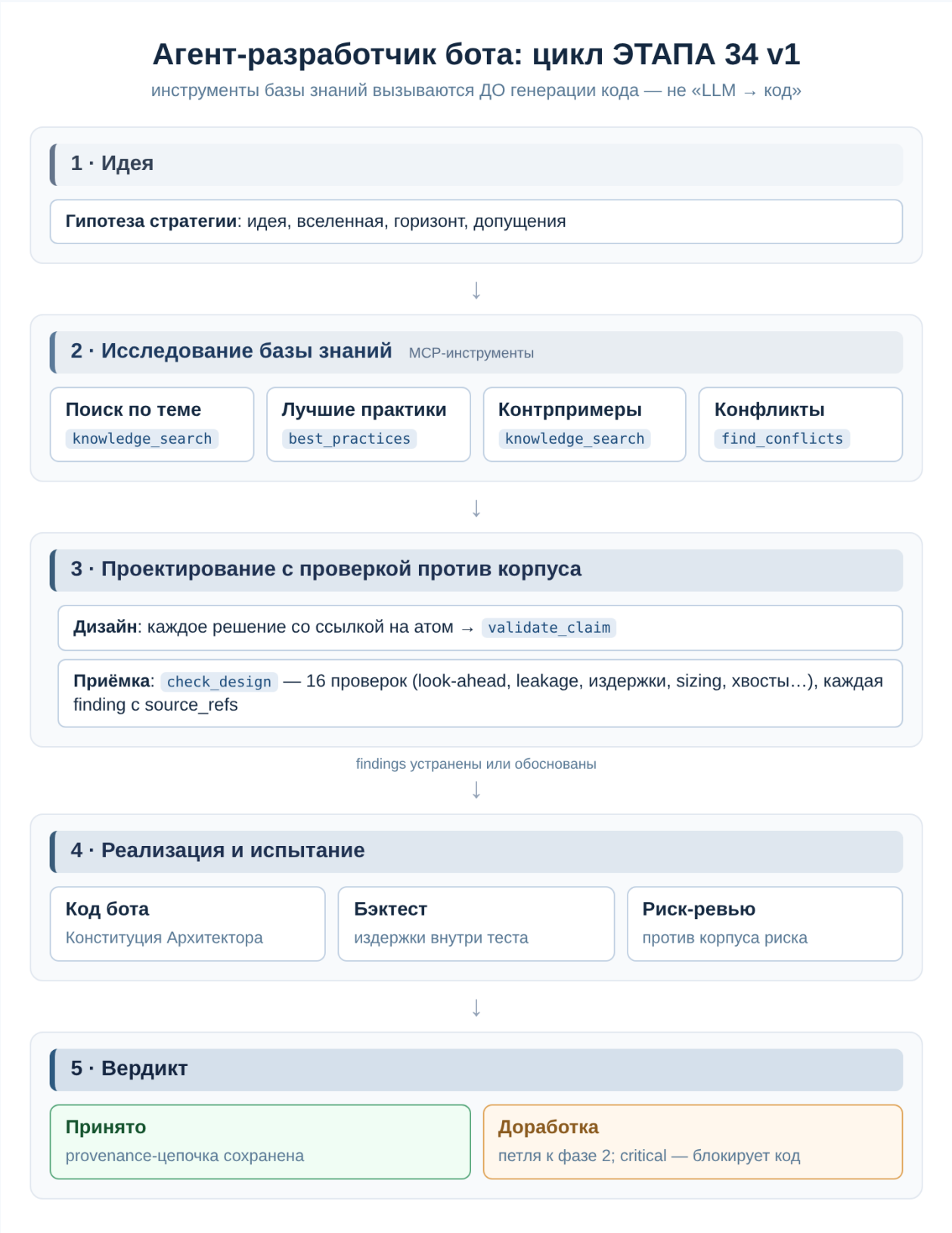
7.2. MCP-контракт (stdio + streamable HTTP)

Один набор инструментов для обоих транспортов: локально — stdio для Claude Code и Cursor (apps/mcp-local), в облаке — streamable HTTP на /mcp (актуальный стандарт MCP; SSE объявлен устаревшим в редакции 2025-03-26, «MCP v2» 2026 года обратно совместим). Инструменты — точное зеркало REST-методов:

Инструмент	Вход (кратко)	Выход
knowledge_search	query, mode, domains, types, temporal_status, top_k	results[] с provenance
knowledge_get	ids[]	карточки целиком
knowledge_trace	ref_id	цепочка claim→atom→chunk→book→page
knowledge_compare	concept, authors[]	agreement / differences / conflicts / evidence
knowledge_find_conflicts	concept	конфликты с классификацией
knowledge_best_practices	concept, asset_class	current/foundational/historical/deprecated + контрпримеры
validate_claim	statement	SUPPORTED / PARTIALLY / UNSUPPORTED / AMBIGUOUS + источники
check_design	design_spec (JSON)	findings[]: severity, claim, source_refs, action

Таблица 6 — MCP-инструменты

`check_design` — центральный инструмент для агентов, пишущих ботов: 16 проверок ЭТАПА 35 v1 (look-ahead bias, data leakage, survivorship bias, overfitting, selection bias, transaction costs, slippage, liquidity, capacity, regime dependence, position sizing, risk of ruin, drawdown, tail risk, parameter instability, validation methodology) — и каждая находка обязана нести `source_refs` на атомы корпуса: проверка опирается на корпус, а не на память модели. Это же замыкает цикл с «Конституцией Архитектора» приложения: курс учит правилам → агенты проверяют проекты через `check_design` по тем же правилам → ментор объясняет провалы по тем же атомам. Полный цикл агента показан на рис. 4.



Существующий канал MENTOR.ask с action=askBooks сохраняется без изменений клиентского API — меняется сторона воркера: вместо пересылки клиентского payload.__rag воркер определяет язык вопроса, вызывает @kc/core.search через adapter-cloud, собирает контекст buildMentorContext с профилем ученика (слабые зоны — из CryptoMentorRAG.memory), подставляет результат в системный промпт и возвращает ответ с pack_version. Офлайн или при недоступности serving клиентский движок v10.8 продолжает работать как сейчас — приложение деградирует изящно, как уже делает с живыми данными.

8 · Стек: решения и верификация

8.1. Верифицированные факты (дата проверки: 29.08.2026)

№	Факт	Значение
1	FTS5 в Cloudflare D1	поддерживается; известный баг — wrangler d1 export падает на virtual tables → бэкапы через build
2	Лимиты D1	10 GB на БД (Paid, жёсткий) / 500 MB (Free); 5 GB на аккаунт (Free); 10 БД на Free
3	Vectorize: размерность	максимум 1536 измерений, float32 → bge-m3 (1024d) помещается
4	Vectorize: metadata	до 10 индексируемых полей; до 64 байт индексируемых данных на вектор; до 10 KiB метаданных
5	Vectorize: namespaces	50 000 на индекс (поднято с 100 в апреле 2026); вектор принадлежит одному namespace
6	Workers AI: bge-m3	@cf/baai/bge-m3: 1024d, 60k токенов контекста, \$0.012/М входных
7	Workers AI: перанкер	bge-reranker-base существует (\$0.0031/М), но не мультиязычный; запрос на v2-m3 открыт
8	sqlite-vec	чистый C без зависимостей, Windows/macOS/Linux, MIT/Apache-2.0; в D1 недоступен
9	MCP: транспорт	streamable HTTP — актуальный стандарт; Cloudflare MCP v2 обратно совместим
10	llama.cpp: rerank	эндпоинт /v1/rerank поддерживается; bge-reranker-v2-m3 GGUF доступен

Таблица 7 — проверенные факты платформы (источники — приложение E)

Каждый факт датирован; агент-разработчик обязан перепроверить пункты 1, 3, 4 и 7 перед реализацией (правило VERIFY: платформа меняется).

8.2. Решения по рекомендациям второго агента

Рекомендация	Решение	Обоснование
FTS5 из коробки вместо JS-BM25, один SQL для D1 и SQLite	принято	факт №1; уход от двух кодовых баз лексического поиска
Namespaces Vectorize для доменов	скорректировано	домены — в metadata-фильтры (\$in, кросс-доменные запросы); namespaces — изоляция prod/dev
bge-reranker-base для слияния каналов	скорректировано	факт №7: модель EN-only → 4 режима реранка, выбор по языку и бюджету
bge-m3 в Workers AI для эмбедингов	принято	факт №6; 1024d укладывается в лимит 1536
sqlite-vec локально	принято	факт №8: работает на Windows, чистый C, активно развивается
Канонические записи в D1/SQLite, не в метаданных Vectorize	принято	метаданные вектора — только для фильтрации; полная запись — в БД по ref_id
Один TS-слой, два тонких адаптера	принято	идентичность логики релевантности — условие валидности eval
search() как инструмент агентов	принято и расширено	оба входа: MCP (stdio + HTTP) и REST; полный набор методов ЭТАПОВ 28 и 63 v1

Таблица 8 — решения по рекомендациям второго агента

9 · Фазировка

Phase 0 — MVP и «битва движков» (2–3 недели)

Скоуп: схема (раздел 5) → импорт 302 атомов PSY_RAG (приложение D) → build трёх книг (одна количественная, одна по риску, одна по психологии — ЭТАП 50 v1) → @kc/core.search (BM25 + векторы локально) → apps/api-local и apps/mcp-local (stdio) → 50 golden-тестов (retrieval, citation, hallucination) → битва с текущим CryptoMentorRAG v10.8 на тех же 302 атомах.

Критерий приёмки Phase 0: Recall@5 нового движка не ниже текущего на golden set — иначе усложнение не оправдано; ветка Zero-Hallucination достижима (запрос «борщ» возвращает «в базе указаний нет»); trace() восстанавливает путь атом→книга для 100% атомов; сборка пака детерминирована.

Phase 1 — Облако (2 недели)

Скоуп: adapter-cloud и apps/worker (REST) → деплой D1 (миграции) и Vectorize (два индекса, namespace dev → prod) → конфиг реранка во всех режимах с замерами → интеграция ментора (раздел 7.3) → contract-тесты: одни golden-тесты гоняются через оба адаптера.

Критерий приёмки Phase 1: расхождение выдачи адаптеров на golden set не более 5% по составу top-5; p95 поиска без реранка — до 400 мс, с llm-rerank — до 3 с; ментор отвечает с rack_version и серверным контекстом; офлайн-фолбэк работает при заглушенном воркере.

Phase 2 — Исследовательский контур для агентов (3–4 недели)

Скоуп: claims/temporal/conflicts API в полном объёме (ЭТАПЫ 31–33, 39 v1) → check_design (16 проверок, связка с Конституцией Архитектора) → MCP streamable HTTP → 30 тестов «1999 против 2026» (ЭТАП 44) → системный промпт агента-конструктора с циклом ЭТАПА 34.

Критерий приёмки Phase 2: агент в Claude Code проходит сценарий «спроектируй стратегию» с не менее чем 5 вызовами knowledge-инструментов до первого файла кода; check_design на заведомо дефектной стратегии находит look-ahead и отсутствие издержек с source_refs; temporal-тесты — не ниже 90%.

Phase 3 — Раздел 2 v1: статьи и индустрия (по готовности)

Скоуп: мониторинг arXiv, SSRN и индустриальных источников (ЭТАПЫ 51–54, 64 v1), quality gate, граф цитирования (ЭТАП 58), расширение eval до 700 тестов (ЭТАП 41). Cloudflare Cron Triggers для мониторинга — отдельное решение этой фазы, не блокирующее ничего ранее.

10 · Риски и открытые вопросы

- **R1. Лимит D1 10 GB.** Полный текст 50–100 книг — примерно 200–500 MB, запас двукратный. При росте корпуса с Разделом 2 тяжёлые чанки выносятся в R2, в D1 — метаданные и атомы; решение зафиксировать при достижении 5 GB.
- **R2. EN-only реранкер в облаке.** Закрыт режимами (раздел 6.4); остаётся вопрос качества llm-rerank на llama-3.3-70b для русского — замеряется в Phase 1; при провале облако работает без реранка, локальный рантайм — с полноценным v2-m3.
- **R3. Двойной рантайм — двойной дрейф.** Контролируется contract-тестами (один golden set, оба адаптера, порог расхождения в CI) и тем, что вся логика релевантности живёт в одном пакете @kc/core.
- **R4. Экспорт D1 с FTS5 падает (баг wrangler).** Источник истины — локальный пак; D1 пересобирается миграциями из пака, а не экспортом; ручные правки в D1 запрещены.
- **R5. Устаревание фактов платформы.** Таблица 8.1 датирована; перед Phase 1 агент перепроверяет факты и обновляет docs/facts.md.
- **R6. Качество стемминга.** Snowball RU агрессивен; деградация контролируется golden-тестами на словоформы («позиция / позиции / позиций» — одинаковый MATCH). При необходимости — гибрид с префиксным сравнением из движка v10.2.

Открытые вопросы: целевой SLA облачного поиска с учётом холодного старта воркера; нужна ли выдача на двух языках (RU-карточки для EN-книг уже есть по конвенции PSY_RAG — core_idea русскоязычный); стратегия обновления векторов при смене модели эмбединга — полная пересборка пака или миграция индексов (решение в Phase 1 по результатам замеров).

11 · Часть II. Этапы для автономного агента

Формат наследует «Задание для автономного агента». ЭТАПЫ v1 (1–65) остаются обязательными для build-слоя; здесь — только serving. Нумерация продолжает v1: ЭТАПЫ 66–76. Критические запреты v1 действуют без исключений.

ЭТАП 66. Окружение

Проверь: Node 20+, pnpm, Python 3.11+ с uv, Git, SQLite 3.38+, llama.cpp server с загруженными bge-m3 и bge-reranker-v2-m3 GGUF, wrangler 4+, доступность ключей Cloudflare (D1, Vectorize, Workers AI). Ничего не устанавливай сверх необходимого; зафиксируй версии в отчёте.

ЭТАП 67. Репозиторий

Создай монорепозиторий из раздела 4.1 (pnpm workspace + uv для Python). Настрой vitest и TypeScript strict. КРИТЕРИЙ: pnpm i && pnpm test зелёный на пустой схеме.

ЭТАП 68. Схема и миграции

Прими schema/0001_init.sql. Напиши мигратор с target sqlite|d1: sqlite выполняет всё, включая vec0-блок; d1 — всё кроме vec0 и PRAGMA. КРИТЕРИЙ: схема применяется к локальному SQLite без ошибок; wrangler d1 execute проходит; фикстуры читаются обоими адаптерами.

ЭТАП 69. Импорт 302 атомов PSY_RAG

Извлеки window.PSY_RAG_DATABASE из index_v10.8.html; маппинг полей — приложение D; создай documents-записи 15 книг. НЕ исправляй известные опечатки — данные переносятся как есть, правки следующим билдом. КРИТЕРИЙ: 302 атома в БД; 0 unsupported; linked_lessons у всех разрешаются; search("тильт") находит не менее 5 атомов Тендлера и Дугласа.

ЭТАП 70. Build трёх книг + пак

Выполни ЭТАПЫ v1 4–20 на трёх книгах (количественная, риск, психология — выбор согласуй в отчёте). Экспортёры: pack.sqlite, ndjson (namespace dev), d1-миграции, graph_export.jsonl. КРИТЕРИЙ: пак детерминирован; отчёт по формату v1 с NOT MEASURED для неизмеренного.

ЭТАП 71. Serving Core

Реализуй в @kc/core: normalize(), search() (fts 30 + vector 30 → RRF → фильтры → порог 1.5 → топ-K), buildMentorContext(), trace(), composeNoHitsAnswer(). Ответ всегда несёт pack_version / backend / rerank. КРИТЕРИЙ: покрытие ядра не ниже 85%; ноль зависимостей от рантайм-API.

ЭТАП 72. Локальный рантайм

adapter-local (better-sqlite3 read-only + sqlite-vec + llama.cpp), apps/api-local (Hono), apps/mcp-local (stdio). Реранк — режим local. КРИТЕРИЙ: POST /api/k/search — до 50 мс на корпусе 302 атома + 3 книги; MCP-сервер виден в Claude Code; trace(psy:tnd_001) возвращает путь до книги Тендлера.

ЭТАП 73. Облачный рантайм

adapter-cloud и apps/worker: REST /api/k/* и /mcp (streamable HTTP). Wrangler: D1, Vectorize kc-atoms-prod и kc-chunks-prod (1024d, metadata-индексы, namespaces prod/dev), Workers AI. Деплой: миграции → upsert в dev → golden set против dev → переключение на prod. КРИТЕРИЙ: совпадение top-5 с локальным не ниже 95%; p95 без реранка до 400 мс; факты платформы №1/3/4/7 перепроверены с датой.

ЭТАП 74. Интеграция ментора

Воркер приложения: action askBooks → детект языка → search через adapter-cloud → buildMentorContext с профилем ученика → системный промпт → ответ с pack_version. Whitelist действий и каналы Д-6 сохраняются; клиентский движок не удаляется. КРИТЕРИЙ: онлайн-ответ содержит источники серверного ретривала; при заглушенном воркере приложение не ломается.

ЭТАП 75. Evaluation: golden set и битва движков

Собери не менее 50 golden-тестов (retrieval 25, citation 10, hallucination 10, temporal 5). Раннер: матрица {adapter-local, adapter-cloud} × {rerank: local, cloud, llm, none} плюс обязательная строка — текущий CryptoMentorRAG v10.8. Метрики ЭТАПА 43 v1. НЕ выдумывай значения; неизмеренное помечай NOT MEASURED. КРИТЕРИЙ: новый движок не ниже v10.8 по Recall@5; отчёт приложен.

ЭТАП 76. check_design и агенты

Реализуй check_design (16 проверок ЭТАПА 35 v1, каждая находка — severity + claim + source_refs + recommended_action). Напиши системный промпт агента-конструктора с циклом ЭТАПА 34 и требованием вызывать MCP-инструменты до генерации кода. КРИТЕРИЙ: на стратегии с подсаженным look-ahead возвращается finding severity=critical с source_refs; прогон агента демонстрирует не менее 5 вызовов инструментов до первого файла кода.

Формат отчёта после каждого этапа

формат отчёта

```
ЭТАП: <номер>
Backend: local | cloud | both
Artifacts: <список файлов + sha256>
Tests: <пройдено/всего; расхождение адаптеров, если оба>
Metrics: <только измеренные; остальное NOT MEASURED>
Unresolved issues: <список>
```

Никогда не придумывай статистику. Проверяй факты платформы перед использованием: таблица 8.1 датирована 29.08.2026.

12 • Приложения

Приложение А. Схема БД

Полный DDL — файл `schema/0001_init.sql` рядом с этим документом: 10 таблиц (`documents`, `chapters`, `sections`, `chunks`, `atoms`, `claims`, `concepts`, `relations`, `lessons`, `pack_meta`, `golden_tests`), 2 FTS5-таблицы с триггерами синхронизации, `vec0`-блок для `sqlite-vec` (включается только для `target=sqlite`) и 2 view (`v_atom_cards` — карточка атома одним запросом; `v_trace_path` — `provenance`-путь для `trace()`). Схема проверена применением к чистому SQLite: 25 объектов, ноль ошибок.

Приложение В. JSON-схемы MCP-инструментов (фрагмент)

MCP-инструмент `knowledge_search`

```
{
  "name": "knowledge_search",
  "description": "Гибридный поиск (BM25+векторы) по корпусу
    книг и статей CryptoNavigator с полным provenance.",
  "inputSchema": {
    "type": "object",
    "properties": {
      "query": {"type": "string"},
      "mode": {"enum": ["atoms", "chunks", "both"]},
      "domains": {"type": "array", "items": {"type": "string"}},
      "knowledge_types": {"type": "array", "items": {"type": "string"}},
      "asset_classes": {"type": "array", "items": {"type": "string"}},
      "temporal_status": {"enum": ["current", "historical", "superseded", "any"]},
      "knowledge_level": {"enum": ["SOURCE", "DERIVED", "APPLICATION",
        "EXPERIMENTAL", "any"]},
      "top_k": {"type": "integer", "minimum": 1, "maximum": 30}
    },
    "required": ["query"]
  }
}
```

Схема `check_design` принимает `design_spec` с полями `hypothesis`, `universe`, `entry_logic`, `exit_logic`, `position_sizing`, `costs_model`, `validation_plan` (обязательны `hypothesis`, `entry_logic`, `exit_logic`) и возвращает `findings[]`. Остальные схемы наследуют входы REST-методов и генерируются из `zod`-типов `@kc/core` в `tools/list` обоих MCP-серверов.

Приложение С. Примеры запросов

запросы

```
-- sqlite-vec (локально)
SELECT chunk_id, distance FROM chunks_vec
WHERE embedding MATCH :query_vector AND k = 30;

// RRF-фьюжн (@ksc/core, идентично на обоих рантаймах)
const k = 60;
const scores = new Map<string, number>();
for (const [name, list] of Object.entries({fts: ftsHits, vec: vecHits}))
  list.forEach((hit, rank) =>
    scores.set(hit.id, (scores.get(hit.id) ?? 0) + 1 / (k + rank + 1)));
const fused = [...scores.entries()]
  .map(([id, rrf]) => ({ id, rrf }))
  .sort((a, b) => b.rrf - a.rrf).slice(0, topK);
```

Приложение D. Миграция PSY_RAG_DATABASE в atoms (ЭТАП 69)

Поле PSY_RAG v4.2	Поле atoms	Примечание
id	atom_id	префикс: tnd_001 → psy:tnd_001
author / author_ru / book	одноимённые	как есть
provenance.source_file	documents.path	создать документ книги; sha256 не вычислять
provenance.chapter_num / chapter_title	chapter_num / chapter_title	как есть
provenance.verbatim_anchor_quote	verbatim_anchor_quote	как есть
provenance.quote_verified (false)	quote_verified = 0	подпись «пересказ» сохраняется
provenance.quote_status	quote_status	agent_anchored_unverified
provenance.chapter_status	content_note	title_verified / range_verified / thematic_grouping
topic / subtopic / core_idea / author_case / protocol	одноимённые	как есть
keywords / linked_terms / linked_lessons	JSON-колонки	resolveLessonId уже применён в v10.8
н/д	atom_type	principle по умолчанию; типизация — следующим билдом
н/д	level / validation / temporal	SOURCE / pending / current — консервативные значения

Таблица 9 — маппинг полей миграции

Инвариант: перенос 1:1 без смысловых правок; любое улучшение (типизация атомов, верификация цитат) — задача билда, а не миграции.

Приложение E. Источники проверки фактов (29.08.2026)

- developers.cloudflare.com/d1/platform/limits — лимиты D1 (актуальность апрель 2026);
- community.cloudflare.com и публикации 2025–2026 о FTS5 в D1; workers-sdk issue №9519 — бар экспорта;

- developers.cloudflare.com/vectorize/platform/limits (август 2026) — 1536 измерений, 10 metadata-индексов, 64 байта;
- developers.cloudflare.com/vectorize/platform/changelog (апрель 2026) — namespaces 50 000;
- developers.cloudflare.com/workers-ai/models/bge-m3 и [/bge-reranker-base](https://developers.cloudflare.com/workers-ai/models/bge-reranker-base) — модели и цены;
- community.cloudflare.com (апрель 2025) — подтверждение EN-only реранкера;
- github.com/asg017/sqlite-vec — sqlite-vec; community-запрос sqlite-vec в D1 (открыт);
- modelcontextprotocol.io (редакции 2025-03-26, 2025-11-25, 2026-07-28) и blog.cloudflare.com/mcp-v2 — транспорт;
- github.com/ggml-org/llama.cpp №8555 и huggingface.co/gpustack/bge-reranker-v2-m3-GGUF — rerank-эндпоинт.